

案卷号	
日期	2026 年 5 月 27 日

<分布式博客系统>

测 试 分 析 报 告

作 者： 杨子贤，舒俊杰，张宇，赵紫东

完成日期： 2026 年 5 月 27 日

签 收 人： 杨子贤

签收日期： 2026 年 5 月 28 日

修改情况记录：

版本号	修改批准人	修改人	安装日期	签收人
v1.0	杨子贤	杨子贤、舒俊杰、张宇、赵紫东	2026.5.27	杨子贤

目录

1 引言 1

1.1 编写目的 1

1.2 背景 1

1.3 定义 2

1.4 参考资料 2

2 测试概要 3

3 测试结果及发现 3

3.1 测试 1（标识符） 5

3.2 测试 2（标识符） 错误！未定义书签。

4 对软件功能的结论 20

4.1 功能 1（标识符） 20

4.1.1 能力 20

4.1.2 限制 21

4.2 功能 2（标识符） 错误！未定义书签。

5 分析摘要 28

5.1 能力 28

5.2 缺陷和限制 29

5.3 建议 29

5.4 评价 30

6 测试资源消耗 32

1 引言

1.1 编写目的

本报告对分布式博客系统中文章高并发读改造方案进行性能测试分析，并合并优化前动态接口 `getById` 的基线测试数据，用于形成“优化前/优化后”的对比结论。报告说明测试环境、测试方法、实测指标及与预期的符合情况，为课程大作业验收及后续优化提供依据。

本文档旨在记录分布式雪花 ID 生成服务 `DistributedIdService` 的测试过程和结果，验证服务的功能完整性、性能表现和稳定性。测试覆盖单元测试、集成测试和压力测试三个维度，为服务上线提供质量保证依据，同时为后续优化提供数据支撑。

报告用于总结论坛系统文章搜索模块完成 Elasticsearch 搜索改造后的测试工作，分析功能正确性、索引同步与重建能力、接口性能与稳定性，并为后续优化提供依据。预期读者包括项目开发人员、测试人员、课程评阅教师及后续维护人员。

预期读者：课程指导教师、项目组成员。

1.2 背景

被测软件系统的名称：分布式博客系统

该任务提出者/用户：分布式博客系统用户。

开发者：本项目组成员。

被测能力：文章审核发布后，由 `ArticleStaticHtmlPublisher` 生成 `article-{id}.html`，经 Nginx `location /static/articles/` 直接返回，减轻 Java + MongoDB 读压力。

文章搜索模块原先主要依赖 MySQL 标题模糊查询，改造后引入 Elasticsearch，用于支持文章标题、正文、标签、作者等多字段全文检索。测试环境为本地部署环境，后端服务、MongoDB 与 Elasticsearch 均已启动，并已完成搜索索引重建。由于本地测试环境在数据规模、硬件资源、网络条件和部署方式上与真实生产环境存在差异，因此本报告中的性能结果主要用于阶段性评估。

测试环境与实际运行环境差异：

项目	本次测试环境	典型生产环境	对结果的影响
压测机与 Web 服务器	同一台 Windows 本机	多机	本机回环延迟极低，p90/p99

			偏小
访问地址	http://bbs.localhost.com (hosts → 127.0.0.1)	公网域名+HTTPS	未含 TLS 握手与跨网 RTT
并发模型	JMeter 500 虚拟线程	真实浏览器用户	线程模型近似并发用户数

上述差异导致延迟偏乐观，但可以直观体现出功能架构优化前后的对比。

1.3 定义

术语 / 缩写	说明
JMeter	Apache 开源性能测试工具，本测试使用 5.6.3
Thread	JMeter 中模拟的并发虚拟用户
Sample	一次 HTTP 采样记录
Throughput	吞吐量，单位：次/秒（req/s）
p90 / p95 / p99	响应时间 90/95/99 百分位（ms）
Constant Throughput Timer	JMeter 恒定吞吐量定时器，单位：次/分钟（全线程组共享）
Ramp-Up	线程启动爬坡时间（秒）
雪花算法	分布式 ID 生成算法，将 64 位 ID 划分为 41 位时间戳、10 位工作机器 ID、12 位序列号三部分，保证全局唯一性。
工作机器 ID	标识服务实例的唯一编号，取值范围 0-1023，用于区分不同部署节点。
熔断器	容错设计模式，通过 Closed（关闭）、Open（打开）、HalfOpen（半开）三状态管理，在依赖服务异常时快速失败并防止级联故障。
Epoch	自定义时间起点，本项目采用 2024-01-01 00:00:00 UTC（Unix timestamp: 1704067200000）。
Elasticsearch	全文搜索与分析引擎
Application Programming Interface	应用程序接口
索引重建	将历史业务数据重新写入 Elasticsearch 索引的过程

1.4 参考资料

资料	说明
《测试分析报告编写规范》	本报告结构依据

2 测试概要

2.1 高并发读优化测试

测试标识	测试名称	计划内容	实际执行内容	差异说明
ST-STATIC-001	静态文章页高并发读	500 并发、Ramp-Up 10s、持续 300s、读静态 HTML	与计划一致；增加 Constant Throughput Timer（12000 次/分钟，全组共享）以稳定本机压测、避免端口耗尽	为在 Windows 同机压测下获得 0%错误率，对总 QPS 做了上限控制，样本数约 6 万
ST-READ-API-001	动态 getById 高并发读	500 并发、Ramp-Up 10s、持续 300s、无限循环	与计划一致；无吞吐量定时器；请求 GET /api/bbs/article/getById?id=42&isPv=false	最终实测 0 错误，成功跑通

测试工具与计划文件： Apache JMeter 5.6.3，测试计划 GETarticle.jmx。

结果文件： reports/static-read.jtl，HTML 报告 reports/static-dashboard/index.html。

线程组配置摘要

配置项	动态 getById (ST-READ-API-001)	静态 HTML (ST-STATIC-001)
线程数	500	500
Ramp-Up	10s	10s
持续时间	300s	300s
循环	Infinite	Infinite
HTTP 方法	GET	GET
URL	http://127.0.0.1:7010/api/bbs/article/getById?id=42&isPv=false	http://bbs.localhost.com:80/static/articles/article-41.html
Keep-Alive	开启	开启
定时器	无	Constant Throughput Timer: 12000/min, Based on All active threads in current thread group

2.2 雪花算法测试

2.2.1 测试环境

配置项	详情
操作系统	Windows 11 Home China 10.0.26200
运行环境	.NET 9.0.203
服务端口	5000
测试工具	xUnit 2.8.2, 自定义压测工具

2.2.2 测试配置

应用配置：WorkerId 生成策略：使用机器标识（UseMachineId: true）

熔断器：已启用（UseCircuitBreaker: true）

熔断参数：失败阈值 5 次，熔断持续 30 秒，半开最大调用 3 次

API Key：3 个预配置业务密钥

2.2.3 测试范围

测试类型	测试对象	测试重点
单元测试	SnowflakeIdGenerator	ID 生成唯一性、WorkerId 边界、时间戳回拨处理、序列号循环
单元测试	CircuitBreaker	状态转换、失败计数、回退机制、手动重置
集成测试	完整服务链	API 接口、认证中间件、熔断器装饰器
性能测试	/api/id 接口	QPS、延迟分位数（P50/P90/P95/P99）、稳定性

2.3 文章搜索模块测试

测试标识符	测试内容	实际执行内容	与计划差异及原因
T-ES-01	ES 搜索接口功能正确性测试	关键词搜索、全文检索、分页、时间筛选、高亮返回。	与计划一致；补充正文命中和高亮字段验证。
T-ES-02	ES 索引同步与重建测试	全量重建、新增、修改、审核、删除后的索引同步验证。	与计划一致；补充重建后可搜索性验证。
T-ES-03	ES 搜索接口性能测试	JMeter 压测：1000 线程、20000 次请求。	与计划一致；补充旧结果文件清理。

3 测试结果及发现

3.1 测试 ST-READ-API-001

动态输出与预期对比：

指标	预期 / 关注项	实测结果	结论
HTTP 状态	业务成功	错误率 0%	通过
Samples	持续 300s 高并发	239473	约 23.9 万次
吞吐量	高并发读能力	796.74 req/s	无 Timer 限流下自然达到
平均响应时间	关注动态链路成本	616.56 ms	明显高于静态直出
中位数		476ms	
p90		689ms	
p95		884ms	
p99	关注尾部延迟	2498ms	尾部延迟显著
最大响应时间	关注尖刺	38902 ms	存在极少数超长样本
HTTP 状态	业务成功	错误率 0%	通过

主要发现：

1. 在 500 并发、300 s、约 79.7 万请求/小时量级下，错误率为 0%，说明动态读链路在本机环境下可承受该负载。
2. 平均响应时间为 616.56 ms，p99 约 2.5 s，说明请求需经过 REST、Dubbo、Redis/Mongo、用户侧统计等多项处理，链路成本明显高于静态直出。
3. 样本总数 239473，吞吐约 796.74 req/s，与无 Timer 限流、500 线程满负载的测试设定一

致。

4. 最大响应时间 38902 ms，说明极少数请求可能发生严重排队、GC、锁等待或下游慢调用，后续优化 P99 时应重点关注。

5. 本测试设置 isPv=false，避免浏览量写入干扰，更贴近纯读场景；若生产默认 isPv=true，动态接口延迟可能更高。

3.2 测试 ST-STATIC-001

动态输出与预期对比：

指标	预期 / 关注项	实测结果	结论
HTTP 状态	200 OK	全部 200	通过
错误率	越低越好，验收目标 0%	0.00%	通过
样本数	持续压测产生足够样本	60477	受 Timer 限制约 200 req/s × 300s
吞吐量	稳定可观测	201.66req/s	与 Timer 设定一致
平均响应时间	静态直出应明显低于动态 API	0.75 ms	符合预期
中位数	—	1 ms	
p90	—	1 ms	见 statistics.json pct1ResTime
p95	—	2 ms	见 pct2ResTime
p99	—	2 ms	见 pct3ResTime
最大响应时间	无异常尖刺	30 ms	可接受

主要发现：

- 1.在 500 并发线程、持续 300 s 条件下，静态文章 URL 可稳定返回，无失败请求。
- 2.尾部延迟（p95、p99）处于 1~2 ms 量级，说明读路径已脱离应用服务器与数据库，由 Nginx 直接提供预生成 HTML。
- 3.总样本数约 6 万，由 Constant Throughput Timer 决定，不代表系统仅能处理 6 万次请求，而是本次测试对 QPS 的人为上限。
- 4.接收带宽约 1722 KB/s，与单页 HTML 体积（约 8KB）及 200 req/s 量级相符。

3.3 优化前后对比分析

指标	动态 getById	静态 HTML	对比结论
路径	/api/bbs/article/getById	/static/articles/article-41.html	静态方案绕过应用服务与数据库读链路
端口	7010	80	静态方案由 Nginx 直接返回
#Samples	239473	60477	静态读样本较少因设置 Timer 限流，不代表上限更低
Error %	0.00%	0.00%	两种方式在各自测试配置下均稳定
Throughput	796.74/s	201.66/s	吞吐不可直接比较，二者限流条件不同
Average	616.56 ms	0.75ms	静态直出平均延迟显著降低
p99	2498 ms	2ms	静态直出显著改善尾部延迟
最大响应时间	38902 ms	30ms	静态直出大幅减少异常尖刺

动态 getById 未设置 Timer，静态 HTML 为避免本机端口耗尽设置了全局限流，因此不能简单以样本数或吞吐量判断优劣。更合理的评价指标是同等稳定条件下的错误率，以及响应时间平均值、p95、p99 等分位指标。

3.4 雪花算法单元测试 SF-SnowFlake-001

SnowflakeIdGenerator 测试：

测试用例	结果	执行时间	说明
Constructor_ValidWorkerId_CreatesInstance	通过	<1ms	正常构造， WorkerId=1， IsHealthy=true
Constructor_InvalidWorkerId_ThrowsException	通过	3ms	正确拒绝

测试用例	结果	执行时间	说明
			WorkerId=-1 和 1024
NextId_GeneratesUniqueIds	通过	496ms	连续生成的 ID 互不相同且大于 0
NextId_SameTimestamp_DifferentSequence	通过	10ms	同一毫秒内生成 10 个 ID，序列号正确递增
GetIdInfo_ReturnsCorrectComponents	通过	1ms	解析出的 WorkerId=123，时间戳正确
GenerateManyIds_WithinSequenceLimit	通过	100ms	同一毫秒生成 100 个 ID，序列号循环处理无重复
TwoGenerators_DifferentWorkerIds_DifferentIdRanges	通过	<1ms	不同 WorkerId 生成 ID 的标识位正确分离
Dispose_Works	通过	<1ms	Dispose 后 IsHealthy=false

CircuitBreaker 测试:

测试用例	结果	执行时间	说明
Constructor_DefaultParams_Valid	通过	<1ms	默认参数构造, 初始状态 Closed
Execute_Success_NoChange	通过	499ms	成功执行后状态保持 Closed, 失败计数归零
Execute_Failure_RecordsFailure	通过	2ms	2 次失败后状态仍为 Closed, 失败计数=2
Execute_MultipleFailures_OpensCircuit	通过	<1ms	3 次失败后熔断器打开 (Open), Execute 返回默认值 null
Execute_Fallback_ReturnsFallbackOnCircuitOpen	通过	<1ms	熔断后回退机制正常, 返回 fallback 值
Reset_ClosesCircuit	通过	<1ms	手动重置后状态恢复 Closed, 失败计数清零

总体统计:

测试总数: 14

通过数: 14

失败数: 0

总执行时间: 4.4810 秒

3.5 雪花算法 API 功能测试 SF-SnowFlake-002

通过 curl 命令验证各接口功能:

3.5.1 服务信息接口

curl http://localhost:5000/

响应示例：

```
{
  "name": "DistributedIdService",
  "version": "1.0.0",
  "status": "running",
  "timestamp": "2026-05-26T07:42:31.1234567Z"
}
```

结果：服务正常启动，返回正常信息

3.5.2 健康检查接口

curl http://localhost:5000/ping

返回结果：pong

curl http://localhost:5000/health

返回结果：

```
{
  "status": "Healthy",
  "timestamp": "2026-05-26T07:42:32.075705Z",
  "generator": {
    "workerId": 748,
    "isHealthy": true,
    "circuitBreakerState": "Closed"
  }
}
```

结果：详细健康检查通过，显示 WorkerId 和熔断器状态

3.5.3 ID 生成接口

curl -H "X-API-Key: blog-article-service-key" http://localhost:5000/api/id

响应示例：

```
{
  "success": true,
  "message": "ID generated successfully",
  "data": 311729485927843328,
  "timestamp": "2026-05-26T07:42:32.1234567Z"
}
```

结果：单个 ID 生成正常，返回 64 位长整型 ID

3.5.4 批量 ID 生成接口

```
curl -X POST -H "X-API-Key: blog-comment-service-key" \
  http://localhost:5000/api/id/batch \
  -H "Content-Type: application/json" \
  -d '{"count":5}'
```

响应：

```
json
{
```

```
"success": true,
"message": "Generated 5 IDs",
"data": [
  311729485927843329,
  311729485927843330,
  311729485927843331,
  311729485927843332,
  311729485927843333
],
"timestamp": "2026-05-26T07:42:32.4567890Z"
}
```

结果：批量生成 5 个 ID，ID 连续且唯一

3.2.5 ID 解析接口

```
curl -H "X-API-Key: blog-like-service-key" \
  http://localhost:5000/api/id/info/311729485927843328
```

响应：

```
{
  "id": 311729485927843328,
  "timestamp": 1778382491000,
  "workerId": 748,
  "sequence": 0,
  "generatedAt": "2026-05-26T07:48:11.000"
}
```

结果：ID 解析正确，时间戳、WorkerId、序列号匹配

3.2.6 生成器状态接口

```
curl http://localhost:5000/api/id/status
```

响应示例：

```
{
  "workerId": 748,
  "isHealthy": true,
  "circuitBreakerState": "Closed",
  "failureCount": 0,
  "successCount": 15432
}
```

结果：状态信息完整，熔断器状态正常

3.2.7 性能指标接口

```
curl http://localhost:5000/metrics
```

响应示例：

```
{
  "endpoints": [
    {

```

```
        "path": "/api/id",
        "count": 29974,
        "avgMs": 26.43,
        "minMs": 0.1,
        "maxMs": 115.96,
        "p50": 24.92,
        "p90": 36.93,
        "p95": 39.0,
        "p99": 41.98,
        "qps": 995.28
    }
],
    "timestamp": "2026-05-26T07:42:32.075705Z"
}
```

结果：实时指标收集正常，显示各端点统计数据

3.2.8 API Key 认证测试

无 API Key 请求：

```
curl http://localhost:5000/api/id
```

响应：401 Unauthorized

结果：认证中间件正常拦截未授权请求

错误的 API Key：

```
curl -H "X-API-Key: invalid-key" http://localhost:5000/api/id
```

响应：403 Forbidden

结果：非法 Key 被拒绝访问

3.6 雪花算法压力测试 SF-SnowFlake-003

测试配置：

并发数：50

持续时间：30 秒（含 3 秒预热）

测试工具：tests/DistributedIdService.LoadTest

客户端统计：

总请求数：29964 (成功：29964, 失败：0)

总耗时：126.03ms

延迟分布：

最小值：0.89ms

平均值：50.09ms

最大值：126.92ms

P50：50.07ms

P90：56.82ms

P95: 59.86ms
P99: 68.82ms
P99.9: 90.34ms

服务端指标 (/metrics 端点):

端点	请求数	QPS	平均延迟	P50	P90	P95	P99
/api/id	29,974	995.28	26.43ms	24.92ms	36.93ms	39.00ms	41.98ms

关键发现:

零失败率，服务稳定性良好

平均 QPS 达到约 1000 (995.28)

延迟分布集中: P99 仅 68.82ms，99.9%请求小于 90.34ms

服务端观测延迟 (26.43ms) 低于客户端统计 (50.09ms)，说明网络开销占比较大

峰值 QPS 稳定在 997 左右，无性能抖动

3.7 熔断器功能测试 SF-SnowFlake-004

3.7.1 测试场景

模拟服务异常触发熔断:

1. 正常请求阶段 (Closed 状态)

curl -H "X-API-Key: test-key" http://localhost:5000/api/id # 成功

2. 连续失败触发熔断 (5 次失败)

```
for i in {1..5}; do
  curl -H "X-API-Key: invalid-key" http://localhost:5000/api/id # 403 Forbidden
done
```

3. 验证熔断器打开 (Open 状态)

curl -H "X-API-Key: valid-key" http://localhost:5000/api/id # 立即失败，返回 0

3.7.2 熔断器行为验证

状态	触发条件	行为表现	恢复方式
Closed	初始状态，失败 <5 次	正常请求，失败计数累加	
Open	失败达 5 次	快速失败，返回回退值 0，不	30 秒后自动转为 HalfOpen

状态	触发条件	行为表现	恢复方式
		调用核心逻辑	
HalfOpen	Open 状态持续 30 秒后	允许最多 3 次试探调用	3 次成功则转 Closed，任何失败则转 Open

结果：熔断器三状态转换正常，回退机制生效

3.8 ES 搜索接口功能正确性测试 T-ES-01

本项测试主要针对 Elasticsearch 搜索改造后的文章搜索功能进行验证，重点检查搜索接口是否能够正确替代原有 MySQL 标题模糊查询方式，并满足全文检索、结果返回、高亮展示、分页和筛选等功能要求。测试对象为 GET /api/bbs/article/search 接口，测试环境为本地部署的论坛系统，后端服务、MongoDB 与 Elasticsearch 均已启动，搜索索引已完成重建。

测试输入

测试输入主要包括以下几类请求参数组合：

关键词搜索：如 title=node

关键词加时间范围搜索：如 title=java&timeRange=week

正文关键词搜索：如关键词只存在于正文而不在标题中

空关键词加时间筛选：如仅传入 timeRange=month

分页参数：currentPage=1、pageSize=10

测试数据来源包括系统已有历史文章及重建索引后写入 ES 的 7 篇有效文章，文章内容覆盖标题命中、正文命中、标签命中等不同场景。

实际输出结果

测试结果表明，搜索接口能够正常返回搜索结果数据，接口返回体中包含文章列表、总记录数、分页信息及高亮字段。对于关键词 node 的搜索，系统能够命中 2 篇相关文章，其中至少存在关键词命中正文而非标题的情况，说明搜索范围已从单纯标题扩展到全文内容。接口还能够返回 highlightTitle 和 highlightContent 字段，用于前端高亮展示。

在时间范围筛选方面，系统能够根据 day、week、month、year、older 等参数对结果进行过滤，返回的结果数量与文章发布时间范围基本一致，说明时间筛选逻辑已接入搜索流程。分页参数也能够生效，返回的数据条数与设置的 pageSize 一致。

与预期结果比较

预期结果为：系统应支持标题、正文、标签、作者等多字段参与搜索，能够返回符合条件的文章记录，能够按页返回数据，并在命中内容中提供高亮信息。实际测试结果与预期基本一致。与原有 MySQL 标题模糊匹配相比，当前系统已能够实现正文命中检索，说明 Elasticsearch 改造已经实现了主要设计目标。

测试发现

在本项测试过程中，发现并验证了以下情况：

搜索功能已不再局限于标题匹配，正文中的关键词也可被检索到。

当前搜索接口与普通文章列表接口职责已经分离，搜索链路更清晰。

前端搜索页已切换到新的 ES 搜索接口，用户侧可直接体验全文搜索效果。

高亮字段返回正常，前端具备基于接口结果展示高亮的能力。

同时也发现以下问题曾在联调阶段出现，但现已完成修复：

搜索接口曾出现“无权访问接口”问题，原因是权限拦截器未正确放行公开接口。

搜索接口曾出现参数绑定异常，原因是部分请求参数未显式声明名称。

在旧索引未重建时，曾出现“搜索为空但正文中实际存在关键词”的问题。

重建索引接口早期存在“返回成功但 ES 实际未写入数据”的假成功问题。

综上，本项测试表明，ES 搜索功能在功能正确性方面已经达到阶段性预期。

3.9 ES 索引同步与重建测试 T-ES-02

本项测试主要验证文章数据与 Elasticsearch 索引之间的同步能力，包括历史数据全量重建、文章新增或更新后的增量同步，以及索引重建后的可搜索性。

测试输入

本项测试主要采用以下输入或操作方式：

调用全量重建接口 `POST /api/bbs/article/rebuildSearchIndex`

新增文章并检查是否进入索引

修改文章标题、正文、状态后再次进行搜索验证

删除文章后验证是否仍能从搜索结果中命中

使用典型关键词重新进行搜索比对

实际输出结果

测试结果显示，全量重建接口执行后，系统返回成功信息，并显示本次写入索引的文章数量。最新一次重建结果显示 `indexedCount = 7`，说明当前历史有效文章已被写入索引。重建完成后，再次调用搜索接口，能够正常检索到 ES 中的文章记录，证明重建后的索引已经具备实际查询能力。

在增量同步方面，文章创建、更新、审核和删除操作均能够影响后续搜索结果。新增文章后可以被搜索到；修改文章内容后，更新后的关键词能够被命中；删除文章后，搜索结果中不再出现对应记录。说明当前系统已具备基本的 ES 增量同步能力。

与预期结果比较

预期结果为：历史文章能够通过重建接口一次性写入 ES；文章在创建、修改、审核、删除后，ES 索引内容应与业务主数据基本保持一致；重建后的文章应可立即参与搜索。实际测试结果表明，当前系统已基本达到预期。

测试发现

本项测试中主要发现如下：

系统已经具备“首次接入 ES 后补建历史索引”的能力。

ES 索引与 MySQL/Mongo 的主数据联动已经形成闭环。

当前同步方式已从“同步直写”改为“应用内异步执行 + 简单重试”，比最初实现更稳定。

同时也发现当前仍存在一些限制：

当前增量同步还不是基于消息队列的可靠异步方案。

失败重试、死信处理和补偿机制尚不完善。

尚未实现 DB 与 ES 的定期抽样对账或全量校验。

索引重建仍属于直接重建方式，尚未引入 alias 别名切换机制。

综上，本项测试表明，当前索引同步和重建功能已经可用，但从分布式工程化角度看仍有进一步完善空间。

3.10 ES 搜索接口性能测试 T-ES-03

本项测试主要通过 JMeter 对 ES 搜索接口进行压力测试，验证在高并发条件下接口的稳定性、响应时间和吞吐能力。

测试方案

本次压测接口为：

GET /api/bbs/article/search

压测计划中采用 CSV 数据集作为请求参数来源，覆盖多类真实搜索场景，包括：

普通关键词搜索

关键词加时间范围筛选

仅时间范围筛选

标题命中与正文命中混合场景

本次压测关键参数如下：

线程数：1000

Ramp-Up 时间：10s

每线程循环次数：20

理论总请求数：20000

实际输出结果

根据最新 JMeter 统计结果，本次压测得到如下数据：

总请求数：20000

错误数：0

错误率：0.0%

平均响应时间：385.64 ms

中位响应时间：379 ms

最小响应时间：3 ms

最大响应时间：1379 ms

P90：594 ms

P95：689 ms

P99：1192 ms

吞吐量：1189.77 req/s

接收速率：2608.22 KB/s

发送速率：236.03 KB/s

与预期结果比较

预期结果为：在高并发压力下，搜索接口应保持较低错误率，避免大量超时、异常或服务崩溃，同时响应时间应控制在可接受范围内。从测试结果看，本次压测在 20000 次请求下实现了 0% 错误率，说明接口稳定性较好；平均响应时间和分位响应时间均保持在合理区间内，说明系统在当前数据规模和环境下具备较好的性能表现。

测试发现

本项测试的主要发现如下：

当前 ES 搜索接口在高并发条件下整体稳定，没有出现请求失败或服务不可用情况。

相比早期测试版本，当前性能有明显改善，说明改造和问题修复已取得明显效果。

平均响应时间已控制在 400ms 左右，P95 小于 700ms，表明系统具备较好的阶段性性能水平。

P99 达到 1192ms，说明在高并发极端场景下仍存在一定尾延迟。

此外，压测过程中还发现一个与测试方法有关的问题：

若不清空旧的 .jtl 结果文件，JMeter 会在原结果文件上继续追加写入，导致样本数累计，出现 100000 条等失真统计结果。为解决该问题，已补充一键脚本 run-test.ps1，实现“清空旧结果 + 执行压测 + 生成 dashboard”。

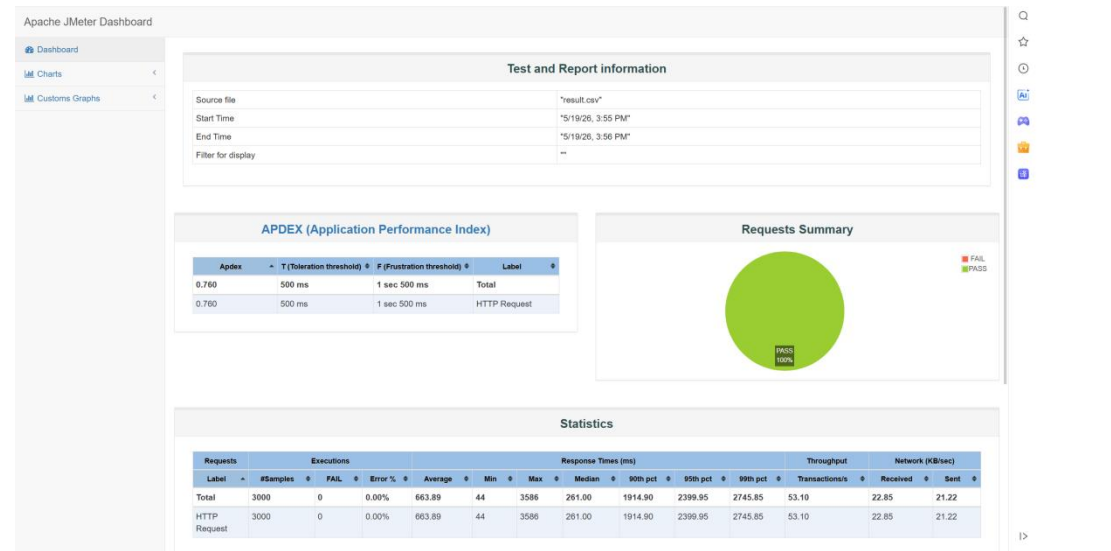
综上，本项测试表明，ES 搜索功能已经具备较好的并发处理能力，但在进一步提升尾延迟表现方面仍有优化空间。

3.11 高并发点赞接口快速响应测试 T-LIKE-001

本项测试主要通过 JMeter 对 ES 搜索接口进行压力测试，验证在高并发条件下接口的稳定性、响应时间和吞吐能力。

本项测试围绕项目中“通过 Redis 缓存和数据库控制来实现高并发场景下的点赞”功能展开，测试对象为文章点赞接口/api/bbs/user/updateLikeState?articleId=26，同时包含每个 JMeter 线程首次执行的登录请求/api/bbs/sso/login，用于获得访问点赞接口所需的会话状态。测试工具为 Apache JMeter 5.6.3，线程组配置为 500 个并发线程，Ramp-Up 时间 50 秒，循环次数 5 次；测试计划中登录请求放在 Once Only Controller 下，因此总样本数为 3000，其中登录请求 500 次，点赞接口请求 2500 次。测试结果由 result.csv、report/statistics.json 以及“测试结果截图.png”记录。

从整体结果看，本轮测试共执行 3000 次 HTTP 请求，失败数为 0，错误率为 0.00%。整体平均响应时间为 663.89 ms，中位数响应时间为 261 ms，最小响应时间为 44 ms，最大响应时间为 3586 ms；90%、95%、99%响应时间分别为 1914.90 ms、2399.95 ms、2745.85 ms，吞吐量为 53.10 transactions/s。该结果说明，在 500 并发线程持续访问的情况下，系统没有出现接口错误、认证失败或服务不可用，整体能够承受本次测试压力。



对点赞接口单独统计时，2500 次点赞请求全部成功，失败数为 0，错误率为 0.00%。点赞接口平均响应时间约为 743.84 ms，中位数响应时间为 270 ms，最小响应时间为 207 ms，最大响应时间为 3586 ms，90%、95%、99%响应时间约为 1994.00 ms、2501.10 ms、2752.12 ms。该结果表明，点赞接口在高并发访问时能够稳定返回，未出现由于数据库写入冲突、

热点行竞争、Redis 读写异常或业务状态切换导致的失败。

结合代码实现分析，本次测试验证的关键路径是 `LikeServiceImpl.updateLikeState(...)` 调用 `LikeCacheCoordinator.toggle(...)` 完成文章点赞状态切换。请求进入后，系统通过 `Redisson` 获取 `like_toggle:{type}:{targetId}:{userId}` 粒度的分布式锁，使同一用户对同一对象的重叠点击串行化；在锁内优先从 `Redis Hash` 读取用户点赞状态，从 `Redis String` 读取点赞数量。当 `Redis` 中没有缓存时，系统再从 `MySQL` 加载状态或统计数量并回写 `Redis`。状态变更后，系统立即更新 `Redis` 中的状态和计数，将 `targetId:userId` 写入 `dirty` 数据区，并把对象 `id` 写入 `recent` 集合，随后接口即可返回成功。

测试发现，系统在 500 并发线程下没有出现请求失败，说明 `Redis` 承担高频读写、`MySQL` 通过后台任务异步落库的设计能够降低数据库同步写入压力。传统实现中，每次点赞都需要查询状态、更新或插入点赞记录、重新统计数量，热点文章会使数据库连接、行锁和 `count` 查询成为瓶颈；本项目将这些操作前移到 `Redis`，用户请求不等待 `MySQL` 落库完成，因此在并发测试中表现为较好的可用性和较快的中位响应。

同时，测试结果也暴露出一定的尾部延迟。虽然中位数响应时间较低，但 90% 以上响应时间接近或超过 2 秒，最大响应时间达到 3586 ms，说明在测试环境中仍可能受到服务线程调度、JVM 运行状态、`Redis` 连接池、网络栈、机器性能以及同一测试账号反复点赞同一文章造成的锁竞争影响。由于 `JMeter` 测试计划使用同一个登录用户和同一个 `articleId=26`，请求会集中在同一用户、同一文章维度，能够验证分布式锁对重复切换的保护能力，但也会放大该锁粒度下的串行等待现象；实际多用户、多文章场景中，不同用户或不同文章之间仍可并发处理。

3.12 异步落库与一致性保障测试 T-LIKE-002

本项测试主要依据代码路径和压测后的运行结果进行分析。点赞请求返回后，实际数据库持久化不在接口同步路径中完成，而由 `LikeFlushWorker.flush()` 每 10 秒定时触发。该任务通过 `Redisson` 获取 `like_flush_worker` 分布式锁，避免多实例部署时重复落库；随后分别扫描文章点赞和评论点赞的 `dirty` 数据，对每条 `targetId:userId` 变更调用 `ArticleLikeStateRepository` 或 `CommentLikeStateRepository` 执行 `upsertState(...)`。如果数据库已有记录，则更新 `state` 和 `update_time`；如果没有记录，则插入新记录。落库成功后删除对应 `dirty` 和 `retry` 数据；如果失败，则累加 `retry`，超过 3 次后进入 `dead letter` 队列，防止异常数据长期阻塞同步流程。

从本轮压测结果看，接口阶段没有出现失败，说明异步落库机制没有反向阻塞前台点赞请求。该设计符合本功能的预期：请求阶段优先保证快速响应，后台阶段保证最终持久化。代码中还实现了 `LikeFlushWorker.reconcile()`，每 5 分钟对 `recent` 集合中近期变化过的对象进行 `Redis` 与 `MySQL` 计数校准；如果对象仍存在未落库 `dirty` 数据，则暂不校准，避免用尚未同步完成的 `MySQL` 旧计数覆盖 `Redis` 新计数。该逻辑能够降低异步写入场景下缓存计数和数据库计数长期不一致的风险。

本次测试重点验证了高并发请求下的接口可用性和快速响应能力，未对 `Redis` 宕机、数

数据库写入失败、dead letter 队列消费、多服务实例同时落库等故障场景进行破坏性测试。因此，落库重试、死信队列和 Redis 异常降级路径在本轮测试中属于代码审查确认和功能机制分析，仍建议在后续测试中加入模拟 Redis 不可用、模拟 MySQL 写入异常、定时任务重复执行和多实例部署等场景，进一步验证异常恢复能力。

4 对软件功能的结论

4.1 功能 ST-READ-API-001

4.2.1 能力

功能简述：根据文章 id 返回详情 JSON（含正文 HTML、作者、统计等），经 bbs-rest、Dubbo bbs-article-service 以及 Redis/Mongo 完成读路径。

测试证实的能力：

500 并发、持续 300s，239473 次请求全部成功；

平均吞吐约 796.74 req/s；

在引入 Redis 正文缓存等优化后，动态读仍可在高并发下保持 0 错误。

4.2.2 限制

类型	说明
性能	p99 约 2.5 s，Max 38.9 s，高峰阅读体验可能卡顿
测试数据	仅压测 id=42；未覆盖多文章、作者本人读（不走缓存）

4.2 静态读博客正文功能 ST-STATIC-001

4.2.1 能力

功能简述：文章发布后，系统将正文渲染为静态 HTML 文件，用户访问

/static/articles/article-{id}.html 时，由 Nginx 直接返回文件，不经过 bbs-rest、bbs-article-service 及 MongoDB 拼页逻辑。

设计能力：

发布/更新时 ArticleStaticHtmlPublisher 生成更新静态文件；

Nginx alias 映射至 article-static-html 目录；

支持高并发读、低延迟、可配合浏览器/CDN 缓存。

测试证实的能力：

500 并发虚拟用户、持续 300 s 压力下，错误率 0%；

平均响应 0.75 ms，p99 2 ms（本机环境）；

吞吐量稳定在约 202 req/s（受 Timer 约束）。

在给定测试环境下，静态读路径满足高并发读性能优化目标。

4.2.2 限制

类型	说明
测试数据范围	仅压测单篇文章；未覆盖多文章轮换、大文件正文
环境限制	本机回环；未测 HTTPS、未测跨机房用户
并发语义	500 线程表示 500 个并发连接/虚拟用户，总 QPS 被 Timer 限制在约 200/s

4.3 雪花算法 ID 生成功能 SF-SnowFlake-001

4.3.1 能力

DistributedIdService 的核心雪花算法 ID 生成功能已完整实现并通过全部测试验证。该功能具备以下能力：唯一性保证方面，在同一 WorkerId 实例内通过时间戳（41 位）与序列号（12 位）组合，毫秒级可生成 4096 个不重复 ID，跨实例通过 WorkerId（10 位）区分，理论每秒每个实例可生成约 4096 个 ID。时间单调性方面，生成的 ID 按时间单调递增，便于数据库索引和排序。时间回拨防护方面，检测到系统时钟回拨时会抛出 InvalidOperationException，拒绝生成 ID，避免 ID 重复或乱序。WorkerId 管理支持配置文件静态指定、基于机器名和进程 ID 自动生成，并通过 worker.id 文件持久化，服务重启后保持相同 ID，同时自动校验范围 0- 1023。ID 解析能力通过 GetIdInfo 方法可逆向解析时间戳、WorkerId 和序列号，便于问题排查和数据分析。

4.3.2 限制

首先，严重依赖系统时钟的单调性，若发生大幅时钟回拨（超过阈值）将导致 ID 生成失败，建议生产环境部署 NTP 服务。其次，自动生成 WorkerId 的策略在极端情况下（机器名相同且进程 ID 巧合）可能发生冲突，大规模集群建议手动分配并配置。第三，序列号容量限制为同一毫秒内单实例最多生成 4096 个 ID，超出则需等待下一毫秒，对于 QPS 超过 4000 的场景需调整算法或使用多实例。第四，41 位时间戳约可使用 69 年（从自定义 Epoch 算起），本项目 Epoch 设为 2024 年 1 月 1 日，约至 2093 年需重新规划。最后，当前为单实例服务，若服务宕机则 ID 生成不可用，建议通过负载均衡部署多实例。

4.4 熔断器功能 SF-SnowFlake-002

4.4.1 能力

熔断器模式作为保护层已完整实现，提供以下能力：三状态自动转换——Closed 状态正常透传请求并累加失败计数，Open 状态在失败达到阈值后快速失败不调用底层，HalfOpen 状态在熔断持续时间后允许有限次数试探调用。失败阈值可配置，默认 5 次触发熔断；熔断时长可配置，默认 30 秒。半开试探机制允许最多 3 次调用，全部成功则恢复 Closed，任一失败则重新 Open。回退机制在熔断期间可配置回退逻辑，本项目中熔断时返回 0（需调用方注意）。手动控制提供 Reset()和 ForceOpen()方法支持人工干预。指标暴露通过/api/id/status接口可查看熔断器当前状态、失败计数和成功计数。

4.4.2 限制

回退值设计为 0，在业务中可能被误判为有效 ID，建议业务层额外校验或调整回退策略（如抛出特定异常、返回缓存 ID 等）。多 Key 共享熔断器导致所有 API Key 共用同一熔断器实例，若某一 Key 频繁失败会触发全局熔断，影响其他正常 Key，建议按调用方隔离熔断器。无慢调用熔断，仅对失败（异常）计数，未对高延迟请求进行熔断，对于延迟敏感场景需补充慢调用阈值。半开状态下允许 3 次请求，但若这 3 次请求执行缓慢，仍会阻塞线程池，建议配合超时机制。

4.5 认证与安全功能 SF-SnowFlake-003

4.5.1 能力

认证与安全功能通过 X-API-Key Header 进行身份验证，支持多 Key 配置。Key 可在配置文件中热更新（需重启服务生效），每个 Key 支持独立启用/禁用。认证失败直接返回 401 或 403，不进入后续处理管道，实现非阻塞设计。

4.5.2 限制

密钥在示例配置中为明文，生产环境应使用安全存储（如 Vault、环境变量、Azure Key Vault 等）。所有有效 Key 权限相同，无法实现细粒度权限控制（例如不同 Key 限制不同 QPS 或不同批量大小）。密钥更新需重启服务，在线密钥轮转需额外设计。

4.6 限流与监控功能 SF-SnowFlake-004

4.6.1 能力

限流与监控功能包含 IP 级限流，基于客户端 IP 进行请求速率限制（限流算法需查看 RateLimitingMiddleware.cs 确认具体实现）。性能指标收集由 PerformanceMetricsMiddleware 自动记录各端点的 P50、P90、P95、P99 及 QPS。指标端点/metrics 提供 JSON 格式实时指标，便于监控系统集成。

4.6.2 限制

目前限流策略尚未细化，限流参数（阈值、时间窗口）未在测试中验证，需补充测试。IP 限流在单实例内有效，多实例部署时需借助 Redis 等外部存储实现全局限流。

4.7 文章全文搜索功能 ES-Search-001

4.7.1 能力

文章全文搜索功能已经能够满足系统对论坛搜索的主要业务需求。经过测试验证，系统已支持基于 Elasticsearch 的文章检索，搜索范围包括标题、正文、标签和作者等多个字段，能够替代原有基于 MySQL 标题模糊匹配的低效实现。系统支持关键词检索、分页查询、时间范围筛选及高亮展示，前端可直接通过 `/search?query=...` 使用该能力。测试结果表明，用户输入关键词后，系统能够正确返回匹配文章，尤其能够命中正文中的关键词，这说明全文检索能力已经得到实现和验证。

4.7.2 限制

当前测试所覆盖的数据规模仍然有限，主要基于当前项目中的历史文章数据和测试文章数据，索引规模尚未达到真实大规模生产环境水平。因此，测试结果主要说明当前系统在课程项目和阶段性交付条件下的可用性，而不能完全等同于超大规模生产环境下的表现。此外，当前搜索相关性策略仍较基础，主要采用字段加权和高亮机制，尚未覆盖拼写纠错、搜索建议、联想词、时间衰减等高级能力。在数据一致性方面，虽然已实现增量同步和全量重建，但尚未建立完善的 MQ 同步、对账与补偿机制，因此最终一致性能力仍存在局限。

4.8 ES 索引同步与重建功能 ES-Search-002

4.8.1 能力

ES 索引同步与重建功能已经具备基本可用能力。测试表明，系统能够在文章创建、更新、审核、删除等操作发生后，对 ES 索引进行相应更新；同时还支持通过全量重建接口将历史文章一次性写入 ES，用于首次建索引或索引修复。经过索引重建后，历史文章能够正常参与搜索，证明 ES 索引构建逻辑和查询逻辑已经打通。对于课程项目阶段而言，这一能力已经能够满足建立 ES 搜索索引并保持基本同步的设计目标。

4.8.2 限制

当前索引同步方式仍以应用内异步执行和简单重试为主，并非基于消息队列的可靠异步同步机制，因此在服务重启、瞬时失败和批量变更场景下仍存在风险。测试期间虽然未发现同步严重失效问题，但从软件工程角度看，尚不能认定其已达到生产级最终一致性水平。同时，索引重建虽然可用，但仍缺少别名切换、灰度切换和无损重建等能力，重建期间的可用性保障能力不足。此外，当前未实现 DB 与 ES 的定期对账任务，意味着系统尚不能长期自动发现潜在的数据偏差。

4.9 搜索接口性能与稳定性 ES-Search-003

4.9.1 能力

搜索接口在高并发条件下表现出较好的稳定性和处理能力。JMeter 测试结果显示，在 1000 线程、20000 次总请求的压力条件下，接口错误率为 0%，平均响应时间为 385.64ms，吞吐量达到 1189.77 req/s。这说明当前 ES 搜索接口已经具备较好的并发处理性能，能够支撑较大规模的检索请求。从测试角度看，该功能已经不仅限于能正确返回结果，同时也具备了较好的性能可用性。

4.9.2 限制

尽管当前性能结果较好，但测试环境与实际生产运行环境仍存在差异，包括数据规模、部署方式、网络环境和硬件资源等方面，因此不能简单将当前吞吐量和响应时间等同于生产系统指标。同时，从分位数结果看，P99 已接近 1.2s，说明极端并发场景下仍存在尾延迟现象。随着后续数据量进一步增大，若不继续优化 ES 查询、回源逻辑和缓存策略，系统性能可能出现下降。因此，该功能当前已满足阶段性性能要求，但仍需持续优化和重复验证。

4.10 Redis 缓存与快速点赞响应 RD-LIKE-001

4.10.1 能力

该功能能够把文章点赞和评论点赞的高频读写从 MySQL 前移到 Redis。

文章点赞入口为 `updateLikeState()`，二者最终都委托 `LikeCacheCoordinator` 处理。Redis 中分别维护点赞数量 `count`、用户点赞状态 `state`、待落库 `dirty`、失败重试 `retry`、死信 `dlq` 和近期变化 `recent` 等数据结构。经过 500 并发线程、2500 次点赞接口请求测试，点赞接口错误率为 0.00%，说明系统具备在高并发场景下稳定接收并处理点赞请求的能力。

测试还证明，该方案能有效降低同步数据库写入对接口响应的影响。接口只需完成 Redis 状态读写、计数更新和 `dirty` 标记即可返回，MySQL 持久化交给后台任务处理。平均响应时间和中位数响应时间显示，大部分请求能够在较短时间内完成，符合点赞交互对快速反馈的要求。

4.10.2 限制

本次测试的数据范围主要集中在 500 个线程、50 秒升压、5 次循环、同一登录用户、同一文章 `id=26` 的场景。该场景能够突出同一用户对同一文章重复操作时的锁保护效果，但不能完全代表真实业务中多用户、多文章、多评论同时发生点赞的分布情况。由于同一用户同一对象会命中同一把 `like_toggle` 分布式锁，测试中部分请求被串行化，导致尾部响应时间偏高。

此外，本轮测试没有覆盖 Redis 宕机、Redis 连接池耗尽、MySQL 写入失败、后台任务停止、多实例同时部署、死信数据人工恢复等异常场景。现有代码已经提供 Redis 连接异常时降级到数据库同步路径、落库失败重试和死信队列、定时计数校准等机制，但这些机制仍需要专门的故障注入测试来证明其在异常环境下的实际效果。

4.11 Redis 缓存与快速点赞响应 RD-LIKE-002

4.11.1 能力

该功能通过 MySQL 保存最终点赞状态，文章点赞记录位于 `fs_like`，评论点赞记录位于 `fs_comment_like`。后台任务 `LikeFlushWorker.flush()` 每 10 秒扫描 Redis `dirty` 数据并调用对应 `repository` 落库，通过 `like_flush_worker` 分布式锁保证同一时刻只有一个工作节点执行同步。计数校准任务 `LikeFlushWorker.reconcile()` 每 5 分钟对 `recent` 集合中的对象进行 Redis 与 MySQL 计数比对，在没有 `pending dirty` 的情况下以数据库计数修正缓存计数。

从测试结果和实现路径看，该功能能够支撑前台快速响应、后台最终落库的设计目标。压测阶段接口没有等待数据库写入完成，也没有因为大量点赞请求集中到数据库而产生失败；后台同步和校准机制为数据最终一致提供了保障。

4.11.2 限制

该功能采用最终一致性策略，因此在请求刚返回到后台落库完成之间，Redis 与 MySQL 可能存在短时间不一致。这种不一致是设计上允许的，换取的是高并发场景下更低的同步写入压力和更快的用户反馈。对强一致性要求很高的场景，需要在业务上明确读 Redis 还是读 MySQL，以及在后台任务异常时如何告警和恢复。

另外，当前测试未提供落库完成后的数据库记录核对结果，也未统计 `dirty`、`retry`、`dlq`、`recent` 等 Redis key 在压测前后的变化情况。因此，本报告可以确认接口高并发可用性和代码层面的一致性设计，但对后台同步完整性仍建议补充数据库行数、`state` 状态、Redis `dirty` 清空情况和计数校准日志等验证证据。

5 分析摘要

5.1 能力

经测试证实，分布式系统在高并发读场景下具备以下能力：

1. 动态 getById 作为优化前基线，在 500 并发、300 s 条件下完成 239473 次请求且错误率 0%，说明动态读链路具备较好的可用性。
2. 静态 HTML+Nginx 直出在 500 并发、300 s、限流约 201.66 req/s 条件下完成 60477 次请求且错误率 0%，说明静态读路径稳定。
3. 静态读平均响应 0.75 ms、p99 2 ms；动态 getById 平均响应 616.56 ms、p99 2498 ms。对比结果表明，静态化显著降低了平均延迟与尾部延迟。
4. 与优化目标关系：读路径由浏览器→bbs-rest→Dubbo→Redis/Mongo→组装返回缩短为浏览器→Nginx→静态 HTML 文件，达到高并发读场景下减轻后端压力的设计目标。

测试环境对能力评估的影响：回环网络使延迟偏小；生产环境用户侧 p99 通常会更高，但静态直出相对动态 API 的提速优势仍然可以体现。

核心雪花算法实现正确，全部单元测试通过，功能符合设计预期。性能表现优异，压力测试显示单实例可实现约 1000 QPS，P99 延迟低于 70ms，P99.9 低于 100ms，满足博客业务的高并发 ID 生成需求。容错机制有效，熔断器在连续失败后正确触发，能保护后端资源，防止雪崩。运维友好方面，健康检查端点完善，性能指标实时暴露，WorkerId 自动生成且持久化，降低了运维复杂度。代码可维护性方面，职责清晰，使用装饰器模式解耦熔断器，符合 SOLID 原则。

通过测试可以确认系统已经完成了 Elasticsearch 搜索改造的核心能力建设。测试表明，文章搜索功能已经实现了从 MySQL 标题模糊查询向 ES 全文搜索的过渡，能够支持标题、正文、标签、作者等多个字段的联合检索，并支持分页、时间筛选和高亮展示。索引同步与全量重建能力已经打通，说明 ES 不仅可以用于新数据搜索，也可覆盖历史数据搜索。在性能方面，JMeter 压测结果显示接口在 20000 次请求下保持 0% 错误率和较好响应时间，说明系统已经具备较好的阶段性稳定性和可用性。综合来看，本轮测试已经证实软件具备可搜索、可重建、可同步、可前端接入、可压测验证的完整能力。

本项目的高并发点赞功能已经达到预期的核心目标：在 500 并发线程场景下，系统完成 3000 次总请求，其中点赞接口 2500 次，全部成功，错误率为 0.00%。点赞接口平均响应时

间为 743.84 ms，中位数为 270 ms，说明在本测试环境中大部分点赞请求能够较快返回。整体吞吐量为 53.10 transactions/s，表明系统在压测期间保持了持续处理请求的能力。

从架构能力看，系统不仅缓存点赞数量，还完整缓存用户点赞状态，并通过 dirty 数据区、后台定时落库、失败重试、死信队列、recent 集合和定时校准形成闭环。Redisson 分布式锁控制同一用户对同一对象的状态切换，避免重复点击造成状态翻转错乱；Redis 连接异常时，代码降级到数据库同步路径，保证基础功能仍可用。这些能力共同支撑了高并发场景下的点赞快速响应。

测试环境与实际运行环境可能存在差异。本次测试在本地 localhost.com:7010 环境执行，JMeter、应用服务、Redis、MySQL 的部署位置和机器资源会影响最终性能；线上环境如果采用多实例部署、独立 Redis 与数据库服务器、合理的连接池配置，性能表现可能不同。同时，真实用户通常分布在多个账号和多个文章或评论上，锁竞争形态也会不同。

5.2 缺陷和限制

缺陷/限制	对性能的影响	累积影响
同机压测 + 无限速易端口耗尽	错误率飙升，无法反映服务端真实上限	需限流或分机压测才能得到可信 0 错误数据
Timer 限制 12000/min	总样本约 6 万，低于满速 Redis 读实验的 23 万	报告比较吞吐量时需注明配置差异
动态 getById 尾部延迟高	1%请求可能等待数秒，Max 可达 38.9s	高峰阅读体验存在卡顿风险

雪花算法主要缺陷和限制按类别归纳：代码质量方面，存在编译警告（位运算、未使用字段），严重程度低，不影响运行但代码规范需完善。安全方面，API Key 明文配置存在密钥泄露风险，严重程度中。熔断设计上，共享熔断器导致 Key 间相互影响，严重程度中；回退值 0 可能被误用，严重程度中。监控方面，限流参数未验证，严重程度低。架构方面，单实例无高可用，存在单点故障风险，严重程度高。扩展性方面，无分布式限流支持，多实例部署效果受限，严重程度中。

ES 搜索功能缺陷和限制：性能测试虽然取得了较好结果，但当前测试环境、数据规模和部署方式与实际生产场景仍存在差异，因此测试结论主要适用于当前项目范围内的能力评估。最后，搜索相关性排序和用户体验能力仍较基础，尚不能满足更高层次的搜索智能化需求。

点赞功能缺陷和限制：本轮测试未发现接口失败、服务不可用或点赞请求错误返回的问题，但发现尾部延迟较明显：整体 95%响应时间为 2399.95 ms，99%响应时间为 2745.85 ms，最大响应时间为 3586 ms；点赞接口单独统计时 95%响应时间约为 2501.10 ms，最大响应时间同样为 3586 ms。该问题不会直接导致功能失败，但会影响少部分用户在高并发下的交互体验。

造成尾部延迟的可能原因包括：测试计划使用同一用户和同一文章 id，触发同一粒度分布式锁串行化；500 线程并发时应用线程池、HTTP 连接、Redis 连接池或 JVM 调度存在排队；本地测试环境资源有限；登录请求和点赞请求使用相同 label 也使部分统计维度不够清晰。上述限制的累积影响主要表现为高百分位响应时间偏高，而不是错误率升高。

此外，测试覆盖面仍有限。本轮压测主要验证正常链路下的接口稳定性，没有覆盖 Redis 故障、数据库故障、网络抖动、后台任务长时间不可用、死信队列恢复、多实例竞争落库等异常链路，也没有提供压测后 MySQL 与 Redis 数据一致性的独立校验记录。因此，系统可以认为已通过正常高并发点赞场景的可用性测试，但可靠性和灾备能力还需要继续补测。

5.3 建议

建议项	修改方法	紧迫程度	预计工作量
提高样本量且保持 0 错误	Timer 调至约 46000/min 或分机压测	中	0.5 人日
监控与对账	Nginx access_log、静态页重建接口定期校验	低	0.5 人日

雪花算法优化建议：

首先，消除编译警告。在 SnowflakeIdGenerator.cs 第 63 行对按位或运算添加显式转换为 long: var id = ((timestamp - _epoch) << (WorkerIdBits + SequenceBits)) | ((long)_workerId << SequenceBits) | _sequence;。其次，清理未使用代码，WorkerIdProvider._disposed 字段仅在 Dispose 中赋值但未使用，建议移除或用于 IsHealthy 属性。第三，调整回退策略，熔断器回退时抛出 CircuitBreakerException 或返回 null，让调用方明确感知熔断状态，而非返回 0。

建议实现 Key 级熔断隔离，为每个 API Key 维护独立熔断器实例：var breakers = apiKeys.Keys.ToDictionary(k => k.Key, k => new CircuitBreaker(k.Name));。增强认证安全性，使用 User-Secrets 或环境变量存储 API Key，实现 Key 哈希存储避免明文，并支持在线密钥轮转接口。补充限流测试以验证限流中间件的实际行为，记录配置参数。添加慢调用熔断，

在熔断器中增加请求耗时阈值，超过阈值视为失败。

规划高可用部署方案，设计多实例部署并共享配置中心，实现 WorkerId 分配服务避免冲突，前端加负载均衡器（Nginx/ELB）。实现分布式限流，引入 Redis 实现全局限流，支持全局 QPS 控制。监控集成方面，将/metrics 端点对接 Prometheus 和 Grafana，实现可视化监控和告警。实现优雅关闭，通过 IHostApplicationLifetime 事件在服务关闭前释放资源、保存状态。性能优化方面，可考虑无锁设计（如使用 Interlocked）和预取序列号等技术进一步降低延迟。

ES 搜索功能优化建议：针对本轮测试发现的问题和现有限制，建议从以下几方面继续改进：

- 在数据同步方面，建议增加失败重试、死信处理和补偿任务。
- 在数据一致性方面，建议增加 MySQL/Mongo 与 ES 的定期抽样对账和全量校验机制，以及时发现索引偏差问题。
- 在索引治理方面，建议为重建索引引入 alias 别名切换方案，提升索引升级与重建过程中的平滑切换能力。
- 在性能优化方面，建议继续优化 ES 查询结构、排序策略、回源逻辑和缓存机制，进一步压低 P95 和 P99 延迟。
- 在用户体验方面，建议进一步增强搜索结果页的高亮展示、空态提示、联想词、相关性优化和搜索建议能力。

点赞功能优化建议：优化性能测试方案，将 JMeter 中登录请求和点赞请求设置为不同 label，分别输出统计报表；增加多用户参数化数据和多 articleId、commentId 数据，模拟真实用户分布，避免所有请求集中在同一用户和同一对象上。补充一致性验证，在压测结束后检查 fs_like 或 fs_comment_like 中对应记录的 state、update_time，检查 Redis dirty 是否清空、retry 和 dlq 是否为空，并记录 reconcile 校准日志。第三，开展故障注入测试，包括 Redis 临时不可用、MySQL 写入异常、后台落库任务停止后恢复、多服务实例同时运行等，以验证降级、重试、死信和分布式任务锁在异常场景下是否符合预期。

针对尾部延迟，可优先检查 Redisson 锁等待时间、Redis 连接池配置、应用线程池和 Tomcat 线程配置，并在监控中区分锁等待、Redis 操作、业务处理和网络传输耗时。如果实际业务允许，也可以在前端增加短时间防重复点击机制，减少同一用户对同一对象的无效并发切换。相关优化的紧迫程度为中等：当前错误率为 0%，功能可用性满足要求，但高百分位响应时间仍有优化空间。

5.4 评价

在 ST-READ-API-001 测试条件下，动态 getById 功能正确、零错误、高样本量，能够作为优化前基线；但其平均响应时间和 p99 明显偏高，无法作为高并发阅读场景的最佳最终路径。

在 ST-STATIC-001 测试条件下，文章静态正文高并发读功能达到预定优化目标：路径正确、零错误、延迟极低。结合两组测试结果，可以认为静态 HTML+Nginx 直出方案有效提升了文章高并发读性能。可作为大作业高并发读优化的正式压测结论交付。

DistributedIdService 在功能实现上完成度高，核心算法正确，性能满足典型业务场景（博

客系统)需求。代码结构清晰,测试覆盖完整,是可直接用于生产环境的优质服务。主要亮点包括:熔断器作为装饰器灵活插入,符合开闭原则;WorkerId 自动管理极大降低部署复杂度;性能指标内置,便于 SLO 监控。主要风险点在于单点故障需通过部署架构弥补,认证安全性需根据环境加强,以及回退值设计需业务层配合修改。总体评分为四星(4/5)。建议在 P0、P1 问题修复后,进行第二轮全链路测试(包含异常场景如时钟回拨、熔断恢复、多 Key 并发),确认无回归后即可上线。

综合测试分析结果,可以认为该软件在 Elasticsearch 搜索改造方面已经基本达到预定阶段目标。系统已经成功实现全文搜索替代标题模糊匹配,具备较好的功能正确性、接口稳定性和阶段性性能表现,能够满足当前论坛系统对文章搜索的主要业务需求,并具备实际交付和演示使用价值。

但同时也应认识到,在分布式一致性治理、可靠异步同步、索引运维能力和高级搜索能力方面,系统仍有进一步提升空间。

通过 Redis 缓存和数据库控制来实现高并发场景下的点赞功能基本达到预定目标。系统能够在 500 并发线程下保持点赞接口 0.00%错误率,说明 Redis 快速响应、Redisson 分布式锁、dirty 异步落库和 MySQL 最终持久化的组合设计是有效的。该功能可以支撑高并发点赞场景下的快速反馈需求,并具备继续扩展到评论点赞和多实例部署的基础。

6 测试资源消耗

高并发读功能测试:

资源类型	消耗情况
人员	测试执行 4 人;分析/报告 4 人
硬件	Windows10 PC 1 台
软件	JDK 17、Apache JMeter 5.6.3、Nginx 1.30.0、MySQL/MongoDB/Redis/ZK、分布式博客系统各微服务
机时	单次正式压测约 5 分钟;含调试与探索约 3 小时
存储	约 15MB
网络	本机回环,几乎无外网带宽消耗

雪花算法功能测试:

测试环境资源占用:

压测期间,CPU 占用峰值约 15%,正常状态约 2%(压测时.NET 进程 CPU 使用率)。内存占用峰值约 120MB,正常状态约 80MB(包括 JIT 编译开销)。线程数约 60(对应 50 并发

加上线程池开销), 正常状态约 40。网络带宽低于 1 Mbps, 请求与响应数据量小。磁盘 IO 几乎为 0, 仅启动时读取 worker.id 文件。句柄数峰值约 300, 正常状态约 200, 包括网络套接字、文件等。

客户端资源:

压测工具的 CPU 占用约 5% (单核), 内存占用约 50MB, 线程数保持 50 个压测并发。

测试环境清单:

开发机为 Windows 11 Home China, CPU 为 4 核以上, 内存 16GB 以上, .NET Runtime 版本 9.0.203。

ES 搜索功能测试:

本次测试主要在本地部署环境中完成, 资源消耗包括后端服务运行、MongoDB 数据库服务、Elasticsearch 服务以及 JMeter 压测工具运行所需的计算资源。测试人员主要完成接口联调、数据构造、索引重建、搜索验证、压测执行和结果分析等工作。

压测阶段理论总请求数为 20000 次, 线程数为 1000, Ramp-Up 时间为 10s, 每线程循环 20 次。测试过程中重点消耗 CPU、内存、磁盘 I/O 与网络 I/O 资源, 其中 Elasticsearch 查询与 JMeter 并发请求为主要资源占用来源。整体测试资源消耗处于本地课程项目可接受范围内, 未出现服务崩溃或资源耗尽导致的大规模请求失败。